The Book

This book builds a small language, one piece at a time. Each piece is added because of something the pieces before it cannot say, and each comes with examples that mean something. Most are about the game rock, paper, scissors.

The language has two parts. The **core** is for saying things: things, functions that turn things into other things, and the kinds things come in. **Effects** are for doing things: keeping a score that changes, giving up, trying every possibility, taking turns, talking to the world outside.

On top of a language can sit **claims** about what is written in it: that a function, given a hand, always gives back a hand, say. A claim can be false, so it has to be checked. The kinds things come in are different: they are defined, and a definition says what a word means, so it cannot be false. Claims add nothing to what can be said, only to what we know about it, so this book leaves them out. It is about the language itself.

As we go, we keep a **map** of the language: a few lines that list every form it has, and every rule about them. Most sections add to it. Its lines are written with letters that stand for parts of the language, never with examples, so they are easy to find, and the map alone, read in order, is the whole language. By the end, the map is the whole language on one page, and every way its pieces combine can be read off it.

1. Things

The simplest thing to say is a thing that is just itself. We say what kind of thing it is by listing every thing of that kind:

```
Hand = rock | paper | scissors
```

The `|` reads "or". This **declares** a **type**, `Hand`: a kind of thing. A `Hand` is `rock`, `paper` or `scissors`, and nothing else. `rock`, `paper` and `scissors` are **constructors**: names that stand only for themselves. A type **lists** the constructors declared in it.

Here is another type:

```
Bool = false | true
```

Each type is declared once, and each constructor belongs to one type only.

Anything we write that says something is an **expression**. A constructor is the simplest expression: it says itself, and there is nothing in it to work out.

So far, things only sit there. Nothing turns one into another.

2. Functions

A **function** turns things into other things. As a type is declared with a name, a function is defined with one, and we say what it gives back for each thing it can be given. Each of these is a **case**:

```
answer = (rock) ↦ paper | (paper) ↦ scissors | (scissors) ↦ rock
```

`↦` reads "gives", and `|` again reads "or": rock gives paper, or paper gives scissors, or scissors gives rock. In each case, what is in parentheses is what the case is for, and what comes after `↦` is its **body**: what the case gives back. This function gives back the hand that beats the one it is given.

This **defines** the name `answer`: from now on, `answer` stands for this function. A name given this way is a **defined name**, and each is defined once. When this book rewrites a declaration or a definition, to say it better, the new one replaces the old. The name is how the function is used: wherever we want it, we write `answer`.

To use a function, we **apply** it to an **input**, written after it in parentheses:

```
answer(rock)
```

This is a question: what does `answer` give back for `rock`? To find out, we **carry it out**, in **steps**. Here one step is enough:

```
answer(rock)
    becomes   paper
```

The cases of `answer` are tried in order, and the application becomes the body of the first case that is for `rock`. Nothing is left to work out in `paper`, so `paper` is the answer.

From here on, "comes to" means "becomes, in one step or more":

```
answer(rock)      comes to   paper
```

One more function, which turns each `Bool` into the other:

```
not = (false) ↦ true | (true) ↦ false

not(true)         comes to   false
```

3. The map

We have now written two kinds of expression:

```
rock          a constructor
not(true)     an application
```

To describe them all at once, and not just these examples, we use letters that stand for parts of the language. `e` stands for any expression, `c` for any constructor, and `n` for any defined name:

```
e ::= c                 a constructor
      n(e)              an application
```

`::=` reads "is one of", and each line is one **form**. So this says: an expression is a constructor, or a defined name applied to an expression.

This is the first piece of the map. Its letters are not expressions, constructors or names. Each marks a place where any expression, constructor or name can go.

What we write is declarations and definitions, then one expression to carry out. Using `T` for the name of a type:

```
T = c | …               declares the type T, with its constructors
n = (c) ↦ e | …         defines n as a function, with its cases
```

In a definition, `| …` means "and more cases like the first, as many as needed": a function has one case or more.

A function has no form of its own among the expressions. It is written only in a definition, which gives it a name, and from then on the name stands for it. Nor is a defined name a form on its own: a function is used by applying it, so its name is always written applied. That is why an application is `n(e)`: only a function can be applied, and a function is always a defined name. `rock(paper)` is not an expression at all, and neither is `not` on its own.

The map shows more than the examples so far. Look at the places with an `e` in them. An application is `n(e)`, so its input can be any expression, another application included. And a case's body is an `e`, so a body can be any expression, not just a constructor. This function gives back the hand that a given hand beats, which is the answer to its answer:

```
beaten = (rock) ↦ answer(answer(rock)) | (paper) ↦ answer(answer(paper)) | (scissors) ↦
answer(answer(scissors))
```

Each body is an application, and the input of each is another application. A form with `e` inside it can hold any form, itself included, so forms go inside forms as deep as we like. Two forms combine without end.

`beaten` uses `answer`, which is defined before it. A definition can use the defined names before it, but not its own, nor one defined after it. Section 9 shows what changes when it can.

Besides forms, the map keeps **rules**: what names mean, the order of steps, how carrying out ends, and what types mean. Each rule is one line, marked with what it is about:

```
names    each type is declared once, and each constructor belongs to one type
names    each defined name is defined once, and stands for its function
names    a definition can use only the defined names before it
```

Most sections from here on add to the map.

4. Carrying out

`beaten(rock)` becomes the body of the case for `rock`. The other bodies are not carried out:

```
beaten(rock)
    becomes   answer(answer(rock))
```

That body is not an answer yet: something is left to work out in it, so carrying out goes on. What a function gives back, in the end, is what its body comes to. But the body is an application inside an application. Which is carried out first?

The rule is: an input is worked out before it goes in. So first `answer(rock)`, then `answer` of what it gave:

```
beaten(rock)
    becomes   answer(answer(rock))
    becomes   answer(paper)
    becomes   scissors
```

The hand that rock beats is the answer to the answer to rock: paper, then scissors.

An expression with nothing left to work out is a **value**. An application always has something left to work out, so for now the values are the constructors. Using `v` for any value:

```
v ::= c                 a constructor
```

A function's bodies are expressions too, but nothing happens inside a function until it is applied. Only then is a body carried out, and only the body of the case that is used.

Here is all of carrying out so far. "`e` becomes `e'`" means one step, and `e'` is what `e` becomes:

```
n(v)     becomes   the body of the first case in n's definition for v
```

Where a step happens: in an application, first in its input, until that is a value; then the application takes its own step. So at each point there is one next step at most, and carrying out never has a choice to make.

```
order    in n(e), e is worked out first, until it is a value; then n(v) takes its step
order    nothing happens inside a function until it is applied
```

But writing a case for every input soon says the same thing more than once.

5. Placeholders

`beaten`, from section 3, says the same thing three times:

```
beaten = (rock) ↦ answer(answer(rock)) | (paper) ↦ answer(answer(paper)) | (scissors) ↦
answer(answer(scissors))
```

To say it once, for whatever hand is given, a case can have a **placeholder** where it had a constructor: a name that stands for whatever the function is given.

```
beaten = (a) ↦ answer(answer(a))
```

The case is for any hand, and its body uses `a` where the hand given goes. When the function is applied, the input is put in place of the placeholder:

```
beaten(rock)
    comes to   answer(answer(rock))
    comes to   scissors
```

A placeholder is like the letters on the map, but written in the language. On the map, `e` marks a place for any expression; in a body, `a` marks a place for whatever the function is given. A placeholder means something only in the body of its own case. A name in a case's parentheses is a constructor if it was declared as one, and a placeholder otherwise, and a placeholder may not have the name of a defined name. So in a body, every name stands for one thing.

What a case is for is now a **pattern**, one of two things. Using `p` for any pattern and `x` for any placeholder:

```
p ::= c                 a constructor: matches only itself
      x                 a placeholder: matches anything
```

A value **matches** a pattern when the pattern describes it. Cases are tried in order, and the first that matches is used, so a last case with a placeholder catches everything the cases before it leave. That is how a function says "otherwise":

```
isRock = (rock) ↦ true | (a) ↦ false

isRock(paper)     comes to   false
```

On the map, a placeholder is a new form of expression, and a case's pattern can now be any pattern:

```
e ::= x                  a placeholder                                  new
      c
      n(e)
n = (p) ↦ e | …          defines n as a function                        (c) widened to (p)
```

And applying puts the input in place:

```
n(v)     becomes   the body of the first case in n's definition that v matches,
                   with v in place of its placeholder, if the pattern is one        widened

names    in a pattern, a name is a constructor if it was declared as one, and otherwise a
placeholder
names    a placeholder means something only in the body of its own case
names    no placeholder has the name of a defined name
names    in an expression, a name is a placeholder of the case it is in, or else a
constructor or a defined name
```

6. Several inputs

Some questions are about two things at once: does one hand beat another? A function can take several inputs, separated by commas, and then each case has one pattern for each:

```
beats = (rock, scissors) ↦ true | (scissors, paper) ↦ true | (paper, rock) ↦ true | (a, b) ↦
false
```

Rock beats scissors, scissors beat paper, paper beats rock; otherwise, the first hand doesn't win. Several values match a case when each matches its own pattern, in order:

```
beats(paper, rock)      comes to   true
beats(rock, paper)      comes to   false
```

In the second, only the last case matches. Without placeholders, `beats` would need nine cases, one for each pair of hands. The placeholders of one case are all different: in `(a, a) ↦ a`, we couldn't tell which input `a` means.

Another function of two inputs, which asks whether both are `true`:

```
and = (true, b) ↦ b | (false, b) ↦ false
```

Inputs are worked out one at a time, from left to right, each to a value, before the function takes its step:

```
and(not(false), beats(rock, paper))
    comes to   and(true, beats(rock, paper))
    comes to   and(true, false)
    comes to   false
```

A function can also take no inputs at all, and is then applied to none. Its body waits until then, so it is a question kept for later:

```
later = () ↦ answer(rock)

later()           comes to   paper
```

Here, asking later gives the same answer as asking now. Section 15 shows where it makes a difference.

On the map, functions and applications now take any number of inputs. In `p, …`, the … means "any number of them, even none", and likewise in `e, …` and `v, …`:

```
e ::= n(e, …)            an application                              widened
n = (p, …) ↦ e | …       defines n as a function                     widened
```

The cases of one function all have the same number of patterns: that is how many inputs it takes, and a defined name is always applied to that many. `beats(rock)` is not an expression: `beats` takes two inputs. Applying it puts each input in place:

```
n(v, …)     becomes   the body of the first case in n's definition whose patterns v, … match,
                      with each value in place of its placeholder                  widened

names    the placeholders of one case are all different
names    the cases of one function all have the same number of patterns: how many inputs it
takes
names    a defined name is applied to as many inputs as its function takes
order    in n(e, …), the inputs are worked out from left to right; then n(v, …) takes its
step    widened
```

7. Getting stuck

So far we have only applied functions to what they were made for. What if a function is given something it has no case for?

```
answer(true)
```

No case of `answer` matches `true`, so no step applies, yet something is left to work out. Carrying out is **stuck**: a value is somewhere it doesn't fit, and there is no way on. That is what stuck means in general: something is left to work out, but no step applies.

A function needn't have a case for everything:

```
onlyRock = (rock) ↦ paper
```

`onlyRock` is a function, and it gets stuck only if it is given something other than `rock`.

The map rules out whatever can be seen in the shape of what we write: `rock(paper)` and `beats(rock)` are not expressions. It can't rule out `answer(true)`, which has the same shape as `answer(rock)`. What tells them apart is which values `answer` has cases for, and an input is often known only by carrying it out, as in `answer(not(false))`.

So carrying out can end in two ways: in a value, which is an answer, or stuck, which is not. That `answer` is only ever given a hand, and so never gets stuck, would be a claim about what we write: it could be true or false, and would need checking. Claims are what this book leaves out. Here, getting stuck is simply one of the ways carrying out can end.

```
ends     in a value, which is an answer
ends     or stuck: something is left to work out, but no step applies
```

8. Holding

So far a value is one thing: one hand, one `Bool`. But a round of the game is two hands, one for each player. A constructor can hold things, its **contents**, of the types written after it:

```
Round = round(Hand, Hand)
```

A `Round` is `round` holding two hands. A constructor that holds contents is always written with them, as many as it holds:

```
round(rock, paper)
```

This looks like an application, but it is not one: `round` is not a function, and nothing is carried out. `round` just holds its contents. `round(rock, paper)` is a value: a round in which the first player showed rock and the second paper. Values built from constructors are **data**. Contents are worked out before they go in, like inputs, from left to right:

```
round(rock, answer(rock))      comes to   round(rock, paper)
```

`round(rock)` and `round` on its own are not expressions, just as `beats(rock)` is not: `round` holds two things. Nor is `rock()`: a constructor that holds nothing is written without parentheses.

The type says what a round is: `round` holding two hands. It doesn't stop `round` from holding other things. `round(true, false)` is data too, just not a `Round`, in the same way that defining the even numbers doesn't stop 3 from existing. A type is a definition: it says which values are of its kind, and forbids nothing.

To use data, we take it apart with a pattern: a constructor, with patterns for its contents.

```
won = (round(a, b)) ↦ beats(a, b)

won(round(paper, rock))        comes to   true
```

The pattern `round(a, b)` matches any round, and gives `a` the first hand and `b` the second. The contents of a pattern are patterns, so a pattern can look more than one layer in:

```
draw = (round(rock, rock)) ↦ true | (round(paper, paper)) ↦ true | (round(scissors,
scissors)) ↦ true | (other) ↦ false

draw(round(paper, rock))       comes to   false
```

`round(paper, rock)` doesn't match `round(paper, paper)`, since `rock` doesn't match `paper`; only the last case matches it.

On the map, a constructor can now be declared holding contents, and types appear inside declarations. As a definition uses only the defined names before it, a declaration uses only the types declared before it, as `Round` uses `Hand`, so a type can't yet hold itself. Using `d` for a constructor as it is declared, and `A` for any type:

```
T = d | …                declares the type T, with its constructors     widened from T = c |
…
d ::= c                  a constructor holding nothing
      c(A, …)            a constructor holding contents of types A, …
A ::= T                  a type declared with its constructors          new

names    a declaration can use only the types declared before it
types    a value is of type T when it is one of T's constructors holding nothing,
         or one holding values of the types declared for its contents
```

Data is a new form of expression, with a new value and a new pattern to match it. Each is a constructor with as many contents as it holds:

```
e ::= c(e, …)            data: c with its contents                      new
v ::= c(v, …)            data: c holding values                         new
p ::= c(p, …)            data built by c, with contents matching p, …   new
```

`e` is inside its own new form, so data can hold any expression, and `v` inside its own, so data can hold data. `p` is inside its own new form, so patterns nest. And since a pattern can now hold placeholders deep inside it, applying puts in place of each placeholder what it matched:

```
n(v, …)     becomes   the body of the first case in n's definition whose patterns v, … match,
                      with what each placeholder matched in its place              widened

names    a constructor is written with as many contents as it holds, in an expression or a
pattern
names    a constructor that holds nothing is written without parentheses
order    in c(e, …), the contents are worked out from left to right; then it is the value
c(v, …)
```

9. Repeating

Every type declared so far has only so many values: a `Round` is one of nine. Some kinds of thing have no end. A constructor's contents can be of the very type being declared:

```
Nat = zero | succ(Nat)
```

A `Nat` is `zero`, or `succ` holding a `Nat`; `succ` reads "one more than". So `zero`, `succ(zero)`, `succ(succ(zero))` are zero, one, two, and so on, with no end: `succ(succ(zero))` is a `Nat` because `succ(zero)` is, and `succ(zero)` is because `zero` is. The rounds of a game are built the same way:

```
Rounds = empty | cons(Round, Rounds)
```

Rounds are none, or a round followed by more rounds; `cons` reads "followed by":

```
cons(round(paper, rock), cons(round(rock, paper), empty))
```

A pattern sees only as many layers as it is written with. To work through rounds of any length, a function must go once around for each layer. First, a small function that adds one to a total when a round was won:

```
more = (true, total) ↦ succ(total) | (false, total) ↦ total
```

To go around once for each layer, a function uses itself. A definition can use any defined name, its own included, even one defined after it. `wins` counts the rounds the first player won:

```
wins = (empty) ↦ zero | (cons(this, rest)) ↦ more(won(this), wins(rest))
```

In no rounds, none. In this round followed by the rest: as many as in the rest, and one more if this round was won.

```
wins(cons(round(paper, rock), cons(round(rock, paper), empty)))
    comes to   more(won(round(paper, rock)), wins(cons(round(rock, paper), empty)))
    comes to   more(true, wins(cons(round(rock, paper), empty)))
    comes to   more(true, more(won(round(rock, paper)), wins(empty)))
    comes to   more(true, more(false, wins(empty)))
    comes to   more(true, more(false, zero))
    comes to   more(true, zero)
    comes to   succ(zero)
```

Each time around takes one layer off the rounds, until they are `empty` and the first case ends it. One round was won.

Adding works the same way. Zero and `b` make `b`; one more than `a`, and `b`, make one more than `a` and `b`:

```
add = (zero, b) ↦ b | (succ(a), b) ↦ succ(add(a, b))

add(succ(zero), succ(zero))      comes to   succ(succ(zero))
```

Since a definition can use one defined after it, two definitions can use each other. `even` asks whether a number is even, and `odd` whether it is odd, each by asking the other about one less:

```
even = (zero) ↦ true | (succ(a)) ↦ odd(a)
odd = (zero) ↦ false | (succ(a)) ↦ even(a)

even(succ(succ(zero)))           comes to   true
```

`even` uses `odd`, which is defined after it. Declarations work the same way: a type can hold a type declared after it, and two types can hold each other. In a game where the players take turns, each hand is answered by the other player's:

```
Turn = turn(Hand, Reply) | over
Reply = reply(Hand, Turn)
```

A `Turn` is a hand followed by a reply, or the end of the game; a `Reply` is a hand followed by the next turn. `Turn` holds `Reply`, which is declared after it, and `Reply` holds `Turn`. So in

```
turn(rock, reply(paper, turn(scissors, reply(rock, over))))
```

no player shows two hands in a row, and since only a `Turn` can be `over`, every turn gets its reply.

Nothing makes a definition use itself on something smaller, so repeating need not stop:

```
loop = (a) ↦ loop(a)

loop(true)      comes to   loop(true), and so on, forever
```

So carrying out has a third way to end, which is not to end at all: it can go on forever.

Repeating added no form to the map. It is two uses of forms already there: a type's name inside the contents of its own constructors, and a name inside its own definition. Until now, the rules about names kept both out: a declaration could use only the types declared before it, and a definition only the defined names before it. Repeating widens both rules, and adds a third way to end:

```
names    a declaration can use any type, its own included, even one declared after it
widened
names    a definition can use any defined name, its own included, even one defined after it
widened
ends     or never: carrying out goes on forever
```

10. Functions as values

Until now, a defined name was always applied, and every value was data. From here on, a defined name can also be written on its own, without inputs. It then stands for its function, and it is a value, like a hand: nothing is left to work out in it, since nothing happens inside a function until it is applied. So whatever can be done with a value can be done with a function: it can be an input, be given back, or be held in data.

A function given a function:

```
twice = (f, a) ↦ f(f(a))

twice(answer, rock)       comes to   scissors
```

`twice` takes a function and a hand, and applies the function twice. `answer` is a value, so it goes in as it is: `twice(answer, rock)` becomes `answer(answer(rock))`.

In the body, the placeholder `f` is applied, and that is new: until now, only a defined name could be applied. A function that is given a function has to apply whatever it was given, so an application is now `e(e, …)`: what is applied can be any expression, a placeholder as here, or an application, as in `against(rock)(paper)` below. That has a cost. The map can't see what a placeholder will stand for, so it can't stop it from standing for something that is not a function, or for a function that takes a different number of inputs:

```
twice(rock, rock)         comes to   rock(rock(rock)), stuck: rock is not a function
twice(beats, rock)        comes to   beats(beats(rock)), stuck: beats takes two inputs
```

Written directly, `rock(rock)` and `beats(rock)` are still not expressions, since the name can be seen: `rock` holds nothing, and `beats` takes two inputs. But through a placeholder, only carrying out finds them, and saying they never happen would be a claim.

A function that gives back a function needs more. `against` is to take a hand, `a`, and give back a test: does a hand beat `a`? With definitions alone, each test needs a definition of its own:

```
againstRock = (b) ↦ beats(b, rock)
againstPaper = (b) ↦ beats(b, paper)
againstScissors = (b) ↦ beats(b, scissors)
against = (rock) ↦ againstRock | (paper) ↦ againstPaper | (scissors) ↦ againstScissors
```

This says the same thing three times, as `beaten` did before placeholders, and a function like it that was given a `Nat` would need a test for every number, with no end. A placeholder can't fix it this time. A definition stands alone: its bodies can use its own placeholders and no others, so no test defined on its own can use the `a` that `against` was given.

So a function can also be **written out**, without a name, wherever an expression can go. Written out inside a body, it can use the placeholders of the case around it, since a placeholder means something in the body of its own case, functions written inside that body included:

```
against = (a) ↦ (b) ↦ beats(b, a)

against(rock)             comes to   (b) ↦ beats(b, rock)
against(rock)(paper)      comes to   true
```

A body reaches as far as it can, so the body of the outer function is the whole inner function. A function written out is a value, like a defined name: nothing happens inside it until it is applied. Still, `rock` went in place of `a` inside the inner function: an input goes wherever its placeholder is used, even where nothing is carried out yet. So `against(rock)` gives back a test made from `rock`, one no definition had to write.

`against` takes data and gives back a function; `twice` takes a function and gives back data. A function can also take a function and give back a function. `twofold` takes a function, and gives back the function that applies it twice:

```
twofold = (f) ↦ (a) ↦ f(f(a))

twofold(answer)           comes to   (a) ↦ answer(answer(a))
twofold(answer)(rock)     comes to   scissors
```

`twofold(answer)` is the same function as `beaten`, made from `answer` with no definition of its own.

Applying a function written out takes the same step as applying a name, with the function's own cases:

```
((rock) ↦ paper | (paper) ↦ scissors | (scissors) ↦ rock)(rock)
    becomes   paper
```

as `answer(rock)` does. The parentheses keep the function in one piece: since a body reaches as far as it can, without them the input `(rock)` would seem to belong to the last case. Parentheses can go around any expression this way. They only group: they are not a form of their own. What a definition adds to a function is only its name, so that it can be used again, by other definitions and by itself.

An inner function's placeholder can have the same name as an outer one's. Then, inside the inner function, the name means the nearer placeholder, its own, and the outer input does not go there:

```
((a) ↦ (a) ↦ a)(rock)            comes to   (a) ↦ a
((a) ↦ (a) ↦ a)(rock)(paper)     comes to   paper
```

Data holding a function. A player plays by a rule: given the hand the other player showed last, it gives the hand to show next. To say what a player holds, we need the type of such rules:

```
Player = player((Hand) → Hand)
```

`(Hand) → Hand` reads "from `Hand` to `Hand`". It is a type too, a **function type**: the functions that, given a hand, give back a hand. More exactly, a value is of this type when, applied to any hand, it never gets stuck, and if carrying out ends, it ends in a hand. `answer` is one, and so is `beaten`. `onlyRock` is not: given `paper`, it gets stuck.

```
next = (player(f), a) ↦ f(a)

next(player(answer), rock)      comes to   paper
```

This player always shows what would have beaten the last hand.

A pattern can take data apart, but nothing takes a function apart: no pattern looks inside one. A function given a function can apply it, and nothing else.

On the map, a defined name on its own and a function written out are new forms of expression and new values, and an application can now apply any expression. Applying a function written out puts each input in place, as applying a defined name does:

```
e ::= n                  a defined name                                  new
      e(e, …)            an application                                  widened from n(e, …)
      (p, …) ↦ e | …     a function, written out                          new
v ::= n                  a defined name                                  new
      (p, …) ↦ e | …     a function, written out                          new

((p, …) ↦ e | …)(v, …)     becomes   the body of the first case whose patterns v, … match,
                                      with what each placeholder matched in its place    new
```

And the rules this section added:

```
order    in e(e, …), first what is applied, until it is a value; then the inputs, from left
         to right; then the application takes its step
widened
ends     stuck also when what is applied is not a function, or takes a different number of
inputs
names    a placeholder means something in the body of its own case, functions inside it
included    widened
names    in an expression, a name means the nearest placeholder of that name, or else a
constructor
         or a defined name
widened
names    an input goes wherever the name means its placeholder, functions written there
included
names    a name written right before inputs is given as many as it takes: a defined name, as
many as
         its function takes; a constructor, as many contents as it holds, in an expression or
         a pattern, and never none if it holds some
widened
writing  a body reaches as far as it can
writing  parentheses only group: (e) is e, kept in one piece
```

A function type is a new form of type. Its parts are types, so it can hold any type, a function type included:

```
A ::= (A, …) → B         a function type                                 new

types    a value is of type (A, …) → B when, applied to values of types A, …, it never gets
stuck,
         and if it ends, it ends in a value of type B
```

One thing waits. A type can hold itself in its contents, as `Nat` does, but not yet inside a function type, as in `Test = test((Test) → Bool)`. Reading such a type takes more care, and section 17 gives it.

```
types    a type can hold itself in its contents, but not inside a function type
```

The rest needed nothing new. An input can be any expression, and so can a body, so either can be a function; and the types in `c(A, …)` can be function types. Every value is data or a function, a defined name counting as a function. So every place a value can go holds one or the other:

```
(Hand) → Bool                      takes data, gives data              isRock
(Hand) → (Hand) → Bool             takes data, gives a function        against
((Hand) → Hand, Hand) → Hand       takes a function, gives data        twice
((Hand) → Hand) → (Hand) → Hand    takes a function, gives a function  twofold
Round = round(Hand, Hand)          data holding data                   round(rock, paper)
Player = player((Hand) → Hand)     data holding a function             player(answer)
```

11. Types left open

`Rounds` holds rounds. A list of hands would need another type just like it, with constructors of its own, since each constructor belongs to one type only. The idea is the same each time; only the type of the items changes.

So a type can leave something open. It can take **placeholders** for types, listed after its name, and is filled with that many types wherever it is used:

```
List(X) = empty | cons(X, List(X))
Pair(X, Y) = pair(X, Y)
```

`List(Hand)` is a list of hands: `empty`, or `cons` holding a hand and a `List(Hand)`. `List(Round)` is a list of rounds. `List` replaces `Rounds`: from here on, `empty` and `cons` are `List`'s constructors. `Pair(Bool, Nat)` holds a `Bool` and a `Nat`.

Functions need nothing like this. A function looks only at how a value is built, so one that works through a list works through a list of anything. `wins` counts the rounds in a `List(Round)` as it did before, and `each`, which applies a function to every item, works with any function and any list:

```
each = (f, empty) ↦ empty | (f, cons(a, rest)) ↦ cons(f(a), each(f, rest))

each(answer, cons(rock, cons(paper, empty)))      comes to   cons(paper, cons(scissors,
empty))
each(not, cons(true, empty))                      comes to   cons(false, empty)
```

On the map, `X` stands for any placeholder for a type; in what we write, they are names like `X` and `Y`. As with the placeholders of a case, those of one declaration are all different: in `Pair(X, X)`, we couldn't tell which type `X` means.

```
T(X, …) = d | …          a type that takes types                        widened
A ::= T(A, …)            a type, filled; T alone if it takes none      widened from T
      X                  a placeholder for a type                      new

names    in a declaration, every name in a type is a declared type, or one of the
declaration's
         own placeholders
names    the placeholders of one declaration are all different, and none has the name of a
declared type
names    a type is filled with as many types as it takes
types    a value is of type T(A, …) when it is built by one of T's constructors, holding
values of
         the types declared for its contents, with A, … in place of T's placeholders
widened
```

The `A`s in `T(A, …)` are any types, so types nest too: `List(List(Hand))`, `List((Hand) → Hand)`, `Pair(Round, List(Bool))`.

The core

That is the whole core. Every value is data or a function, and each is made one way and used one way:

	made by	**used by**
data	a constructor, holding values	a pattern, which takes it apart
function	cases, each with patterns and a body	applying it to inputs

Two kinds of name refer to values: a placeholder stands for what a case was given, and a defined name stands for its function, whose definition may use the name itself.

A type says what a kind of thing is: which data, built how, or which functions, giving back what. It is a definition, and nothing in carrying out uses it.

Everything in the core says something, and nothing changes: an expression and what it comes to say the same thing, the second more directly. The rest of the book is about doing.

12. State, written out

As they play, the players keep score: after each round that the first player wins, their score goes up by one. A score changes, and in the core nothing changes. But a change can be said with values: the score before, and the score after.

```
score = (this, total) ↦ more(won(this), total)
```

Given a round and the score before it, `score` gives back the score after it. Two rounds, the first won and the second lost, starting from zero:

```
score(round(rock, paper), score(round(paper, rock), zero))
    comes to   score(round(rock, paper), succ(zero))
    comes to   succ(zero)
```

The score is a **memory**: something kept, and changed as we go. Here it is written out as a value, handed from one step to the next. The inner `score` is an input, so it is carried out first, and the first round is counted first: the order of the rounds is the order in which the memory is handed on.

This works, but it has a cost. This function asks whether the first player won both of two rounds:

```
both = (first, second) ↦ and(won(first), won(second))
```

To keep the score as well, playing a round must give back two things, whether it was won and the new score, in a pair:

```
play = (this, total) ↦ pair(won(this), more(won(this), total))
```

And `both` must be rewritten to take the score, hand it through each round in turn, and give it back with its answer:

```
both = (first, second, total) ↦ ((pair(a, m)) ↦ ((pair(b, t)) ↦ pair(and(a, b), t))
(play(second, m)))(play(first, total))
```

This reads: play the first round with the score `total`, and call what it gives back `a` and `m`; play the second with `m`, and call what that gives back `b` and `t`; give back `and(a, b)`, with the score `t`.

`both` doesn't care about the score at all: it only hands it along. And every function that uses `both` must now do the same, and every function that uses those. The memory goes in and out of every function between the place it starts and the places that use it, and each of their definitions changes. One small idea, a score, touches everything on the way.

What we want is to mention the score only where it is used.

13. Requests and handlers

Take a simpler case of the same problem first. To know whether a hand wins, a function needs the hand the opponent plays. Written out, that hand would be one more input, handed through every function on the way, just like the score. Instead, the function can ask for it:

```
effect Opponent = opponent : () → Hand
```

This declares an **effect**, `Opponent`, with one **request**, `opponent`. Like a function type, `() → Hand` says what the request takes and what it is answered with: it takes nothing, and is answered with a `Hand`. A request is written like an application, always with as many inputs as it is declared with:

```
beatsOpponent = (a) ↦ beats(a, opponent())
```

`opponent()` asks for the opponent's hand, and `beatsOpponent` says whether `a` beats it.

Carrying out `beatsOpponent(rock)` goes as far as this:

```
beatsOpponent(rock)
    comes to   beats(rock, opponent())
```

The next thing to work out is the input `opponent()`, and no rule carries it out. A request is not a value, and takes no step of its own: it waits to be answered. It is not stuck, though: it is waiting for an answer from outside the expression.

A **handler** answers it. It goes around the expression that asks:

```
handle beatsOpponent(rock) with
      (outcome) ↦ outcome
    | (opponent(), k) ↦ k(scissors)
```

A handler has cases, like a function. The first is for the value the expression comes to in the end: here it gives that value back as it is. The second is for the request `opponent()`, and its body answers `scissors`. In a handler's case, a request's name stands for that request, as a constructor's name does in a pattern, and like a defined name, it is never the name of a placeholder. To see how the answer is given, we need to know what `k` is.

When the request is made, the expression around it is waiting for an answer: `beats(rock, opponent())` is waiting for a hand, to put where `opponent()` is. That waiting expression is **the rest** of the computation, and it can be written as a function. Given an answer, it puts it where the request was, and goes on, with the handler still around it:

```
(y) ↦ handle beats(rock, y) with …
```

Here, and from now on, `with …` stands for the same handler's cases, written out again. The request's case receives the rest as its placeholder `k`. So when `opponent()` is reached, the handler takes a step: it becomes the body of its case for the request, with the rest in place of `k`. Applying the rest to `scissors` goes on with `scissors` where the request was:

```
handle beatsOpponent(rock) with …
    comes to   handle beats(rock, opponent()) with …
    becomes    ((y) ↦ handle beats(rock, y) with …)(scissors)
    becomes    handle beats(rock, scissors) with …
    comes to   handle true with …
    becomes    true
```

In the last step, the expression has come to a value, `true`, so the handler becomes the body of its first case, which gives `true` back.

`beatsOpponent` never says where the opponent's hand comes from: the handler does. A handler that answered `paper` would make the same `beatsOpponent(rock)` come to `false`, without changing `beatsOpponent` at all.

On the map, using `F` for the name of an effect, `r` for a request, and `h` for a handler's case:

```
effect F = r : (A, …) → B | …    declares the effect F, with its requests: what each takes,
                                 and what it is answered with
new

e ::= r(e, …)                    a request
new
      handle e with h | …        a handler around e
new

h ::= (p) ↦ e                    for the value the expression inside comes to
      (r(p, …), x) ↦ e           for a request: patterns for its inputs, and a placeholder
for the rest
```

The steps:

```
handle v with h | …     becomes   the body of the first case for values that v matches,
                                  with what each placeholder matched in its place
handle e with h | …     becomes   the body of the first case for r whose patterns v, … match,
                                  with what they matched in place, and the rest in place of
x,
                                  when the next thing to work out in e is a request r(v, …),
                                  and no handler in e around that request has a case for r
```

Otherwise, the expression inside a handler takes its steps as usual. A request that a handler has no case for passes it by, to a handler further out:

```
handle (handle beatsOpponent(rock) with (outcome) ↦ not(outcome)) with
      (outcome) ↦ outcome
    | (opponent(), k) ↦ k(scissors)

    comes to   false
```

The inner handler has no case for `opponent()`, so the outer one answers it. The rest goes on inside both handlers, so the inner one still receives the outcome, `true`, and turns it into `false`.

A handler that has cases for a request, but none that its inputs match, is stuck, as a function is when no case matches: the request is its to answer, and it has no answer. And the rules:

```
names    each effect is declared once, and each request belongs to one effect
names    a request is written with as many inputs as it is declared with, in an expression
         or in a handler's case
names    a request's name stands for that request, in an expression or a handler's case; no
placeholder has it
order    in r(e, …), the inputs are worked out from left to right
order    in handle e with h | …, steps happen in e
rest     the rest is (y) ↦ handle e' with h | …, where e' is e with y where the request was,
         and y is a name used nowhere in e or in the handler's cases
ends     or waiting: the next thing to work out is a request that no handler around it has a
case for
ends     stuck: something is left to work out, no step applies, and it is not waiting
widened
```

Requests and handlers are made one way and used one way, like data and functions:

	made by	used by
effect	a request	a handler, which answers it

14. Ways to go on

The handler above went on once, with one answer. But it holds the rest as a value, `k`, and can do with it whatever a function can do with a function.

It can never go on:

```
handle beatsOpponent(rock) with
      (outcome) ↦ outcome
    | (opponent(), k) ↦ false

    comes to   false
```

The case never applies `k`, so the rest never happens: the handler's answer stands in for the whole computation. This is how errors work. Something asks for what can't be given, and the handler decides what happens instead.

It can go on several times. This handler asks, for each hand the opponent could play, whether `rock` wins, and counts the wins:

```
count = (true) ↦ succ(zero) | (false) ↦ zero

handle beatsOpponent(rock) with
      (outcome) ↦ count(outcome)
    | (opponent(), k) ↦ add(k(rock), add(k(paper), k(scissors)))

    comes to   succ(zero)
```

Each `k(…)` goes on from the request with a different hand, and comes to a value through the handler's first case:

```
k(rock)          comes to   count(false), which comes to zero
k(paper)         comes to   count(false), which comes to zero
k(scissors)      comes to   count(true), which comes to succ(zero)
```

and `add(zero, add(zero, succ(zero)))` comes to `succ(zero)`. Against the three hands, rock wins once. This is search: one question, every answer tried.

So a handler receives a request and the rest, and chooses how to go on: once, never, or several times. The same two forms, request and handler, say all three.

15. State, said once

Now the score:

```
Unit = unit
effect Score = get : () → Nat | put : (Nat) → Unit
```

`get` asks for the score, and `put` sets it. `put` has nothing to give back, so it is answered with `unit`, the only value of `Unit`, which says nothing.

To do one thing and then another:

```
then = (u, a) ↦ a
```

Since inputs are worked out from left to right, `then(e, e')` works out `e` first, then `e'`, and gives back what `e'` gave.

Recording a round:

```
record = (true) ↦ then(put(succ(get())), true) | (false) ↦ false
```

Given that a round was won, `record` adds one to the score and gives back `true`; given that it was lost, it changes nothing. And `both`, rewritten once more:

```
both = (first, second) ↦ and(record(won(first)), record(won(second)))
```

Compare it with the first `both`, `(first, second) ↦ and(won(first), won(second))`. Only `record`, the one place that changes the score, mentions it. No score is handed in or out, and a function that uses `both` doesn't change at all.

A handler holds the memory:

```
keep = (task, start) ↦ (handle task() with
                          (outcome) ↦ ((m) ↦ pair(outcome, m))
                        | (get(), k) ↦ ((m) ↦ k(m)(m))
                        | (put(b), k) ↦ ((m) ↦ k(unit)(b)))(start)
```

`keep` takes a computation kept for later, `task`, and a starting score, `start`. The task has to be kept for later. An input is worked out before it goes in, so a task given directly would make its requests before `keep`'s handler was around it, and nothing would answer them. Kept for later, it starts only at `task()`, inside the handler. Its handler doesn't give back an answer directly: it gives back a function waiting for the score so far, `m`. The `(start)` at the end gives it the starting score; the parentheses before it keep the handler in one piece, so it can be applied. Its cases read:

- when the computation comes to a value: given the score `m`, give back that value together with `m`;
- when asked for the score: given `m`, answer `m`, and go on with the score still `m`;
- when told to set the score to `b`: given `m`, answer `unit`, and go on with the score `b` instead.

The rest, `k`, has the handler around it, so it too gives back a function waiting for a score. That is why what `k(m)` gives back is given a score in turn: `k(m)(m)`.

Two rounds, the first won and the second lost, starting from zero. Below, `k` is the rest after the first request, `get()`, and `k'` the rest after the second, `put(succ(zero))`:

```
keep(() ↦ both(round(paper, rock), round(rock, paper)), zero)
    comes to   ((m) ↦ k(m)(m))(zero)                    the first request is get()
    becomes    k(zero)(zero)
    comes to   ((m) ↦ k'(unit)(succ(zero)))(zero)        the next request is put(succ(zero))
    becomes    k'(unit)(succ(zero))
    comes to   ((m) ↦ pair(false, m))(succ(zero))        the rest comes to false
    becomes    pair(false, succ(zero))
```

Not both rounds were won, and the score is one.

The memory is handed along exactly as in section 12, in and out of every step. But now that is written once, in the handler, instead of in every function on the way.

In the core, the order in which inputs were worked out never changed what they came to. With requests it does. Asked after it is set, the score is one; asked before, it is zero:

```
keep(() ↦ then(put(succ(zero)), get()), zero)                comes to   pair(succ(zero),
succ(zero))
keep(() ↦ ((s) ↦ then(put(succ(zero)), s))(get()), zero)      comes to   pair(zero,
succ(zero))
```

In the second, `get()` is the input of the function written out, so it is worked out first, before `put`. Order has become part of what is said.

`keep` never looks at the memory, only hands it on, so the same handler could keep a hand, or anything else. Only the declaration says that the score is a `Nat`. An effect can take placeholders for types, as a type can. Declared in place of `Score`,

```
effect State(X) = get : () → X | put : (X) → Unit
```

would make `State(Nat)` a memory of a number, and `State(Hand)` a memory of a hand.

On the map:

```
effect F(X, …) = r : (A, …) → B | …    an effect that takes types; F alone if it takes none
widened
```

16. Many memories

`keep` holds one memory. A program often needs more: a score for each player, say, and how many may not be known in advance. Two `keep`s, one around the other, would not do: both answer `get` and `put`, so the inner one would answer every request, and the outer one would never be asked.

So each memory needs a name of its own, which a request can use to say which memory it means. The handler gives each memory a number when it is made, its **place**: the first memory is at `zero`, the next at `succ(zero)`, and so on.

```
effect Memory(X) = new : (X) → Nat | look : (Nat) → X | set : (Nat, X) → Unit
```

`new(a)` makes a memory holding `a`, and is answered with its place. `look(place)` asks what the memory at `place` holds. `set(place, a)` makes it hold `a` instead.

The handler keeps every memory in one list, each at the position of its place. Four small functions work with such a list: how many items it has, the item at a position, the list with the item at a position replaced, and the list with one more item at the end:

```
size = (empty) ↦ zero | (cons(a, rest)) ↦ succ(size(rest))
at = (cons(a, rest), zero) ↦ a | (cons(a, rest), succ(i)) ↦ at(rest, i)
replace = (cons(a, rest), zero, b) ↦ cons(b, rest) | (cons(a, rest), succ(i), b) ↦ cons(a,
replace(rest, i, b))
append = (empty, b) ↦ cons(b, empty) | (cons(a, rest), b) ↦ cons(a, append(rest, b))
```

The handler is `keep` again, with the list of every memory, `t`, where `keep` had one memory, `m`:

```
keepAll = (task) ↦ (handle task() with
                        (outcome) ↦ ((t) ↦ outcome)
                      | (new(a), k) ↦ ((t) ↦ k(size(t))(append(t, a)))
                      | (look(place), k) ↦ ((t) ↦ k(at(t, place))(t))
                      | (set(place, a), k) ↦ ((t) ↦ k(unit)(replace(t, place, a))))(empty)
```

It starts with no memories, `empty`. Its cases read:

- when the computation comes to a value: give back that value;
- when asked for a new memory holding `a`: answer with the next place, which is how many memories there are so far, and go on with `a` added at the end;
- when asked what is at `place`: answer with the item at `place`, and go on with the memories as they were;
- when told to set `place` to `a`: answer `unit`, and go on with the item at `place` replaced by `a`.

Now each player can keep a score of their own. `point` adds one to the memory at a place when a round was won, and `tally` scores one round for both players:

```
point = (true, place) ↦ set(place, succ(look(place))) | (false, place) ↦ unit
tally = (round(a, b), mine, yours) ↦ then(point(beats(a, b), mine), point(beats(b, a),
yours))

keepAll(() ↦ ((mine, yours) ↦ then(tally(round(paper, rock), mine, yours), pair(look(mine),
look(yours))))(new(zero), new(zero)))
    comes to   pair(succ(zero), zero)
```

The two `new(zero)` make two memories, at `zero` and at `succ(zero)`, and the function they are given calls them `mine` and `yours`. Paper beats rock, so `mine` goes up by one, and `yours` stays at zero.

A place is a value like any other: it can be an input, be given back, or be held in data. So two placeholders can hold the same place, and then they name the same memory. Setting it through one changes what the other finds:

```
keepAll(() ↦ ((mine) ↦ ((ours) ↦ then(set(ours, succ(zero)), look(mine)))(mine))(new(zero)))
    comes to   succ(zero)
```

`ours` is given `mine`, so both hold the place `zero`. `set` through `ours` changes the memory there, and `look` through `mine` finds the change.

This is how the names of a language like Python work. Each variable is a memory at a place of its own, so `x = x + 1` is `set(x, succ(look(x)))`. A list is a memory too, and `b = a` puts the same place in `b`: two names for one list, so a change made through `b` is seen through `a`. An object is data holding places, with functions that use them.

Many memories added nothing to the map. `Memory` is an effect like any other, and `keepAll` is written in the language: memory is written with the map, not added to it.

17. Going on later

A handler holds the rest as a value, so it can also keep it in data, and go on later. This is how one computation can be carried out a little at a time, and how two can take turns.

A player can be a computation of its own, which gives its hands one at a time:

```
effect Give = give : (Hand) → Unit

cycle = (a) ↦ then(give(a), cycle(answer(a)))
```

`cycle(rock)` gives rock, then the answer to rock, which is paper, then scissors, then rock again, and so on forever. It is written as if nothing else were happening: give a hand, then go on to the next.

A handler can stop it at each hand, and give back the hand together with the rest, so that whoever holds them can go on when they want the next hand:

```
Moves = done | gave(Hand, (Unit) → Moves)

moves = (task) ↦ (u) ↦ handle task() with
                           (outcome) ↦ done
                         | (give(a), k) ↦ gave(a, k)
```

`moves` takes a player as `keep` takes its task, as a computation kept for later. What it gives back is a function waiting for `unit`. Given it, it starts the player, and stops at the first hand. The rest, `k`, is waiting for `unit` too, the answer to `give`. Given it, it goes on to the next hand, and stops again. So a player not yet started and a player stopped after a hand are the same kind of thing: a function that, given `unit`, goes on to the next hand. Below, `k`, `k'` and `k''` are the rests after each hand:

```
moves(() ↦ cycle(rock))(unit)      comes to   gave(rock, k)
k(unit)                            comes to   gave(paper, k')
k'(unit)                           comes to   gave(scissors, k'')
```

Each hand is worked out only when it is asked for. `cycle` never ends, but it is carried out only as far as someone asks. This is how a generator works: it gives back items one at a time, each when it is asked for.

Two players can take turns. `turns` plays a number of rounds, asking each player in turn for its next hand, and stops early if either has no more:

```
turns = (zero, first, second) ↦ empty
      | (succ(i), first, second) ↦ ((gave(a, first), gave(b, second)) ↦ cons(round(a, b),
turns(i, first, second))
                                     | (a, b) ↦ empty)(first(unit), second(unit))
```

`first(unit)` and `second(unit)` ask each player for a hand. Each comes back as a hand and the player as it stands after it, and the inner function takes them apart. Inside it, `first` and `second` mean its own placeholders, the players after their hands, as in section 10. A player that always shows rock:

```
rocks = () ↦ then(give(rock), rocks())

turns(succ(succ(succ(zero))), moves(() ↦ cycle(rock)), moves(rocks))
    comes to   cons(round(rock, rock), cons(round(paper, rock), cons(round(scissors, rock),
empty)))
```

Three rounds, and `wins` of them comes to `succ(zero)`: only paper against rock was won. After the third round, neither player is asked for another hand.

Each player is written as if it played alone, and keeps track of where it is in its own rest: `cycle` remembers its last hand in `a`. `turns` holds both rests, and decides which goes on, and when. Add a request that means "wait until this is ready", and a handler that holds many rests and goes on with whichever is ready, and this is how `async` and `await` work, and how many computations share one

computer. Carrying out takes one step at a time, so running two computations truly at the same time cannot be said in steps: a program can only ask for it, with a request that the outside answers.

So a handler can go on in four ways: once, never, several times, or later. The same two forms, request and handler, say all four.

`Moves` holds itself in a new way: not directly, as a list holds a list, but inside a function type. A value is a `Moves` when it is `done`, or `gave` holding a hand and a function that, given `unit`, never gets stuck, and if it ends, ends in a `Moves`. That `Moves` is defined by the same sentence, and so on, and for `cycle`, which never ends, the sentence is never finished. So a type that holds itself is read this way: a value is of the type when following the definition, as far as it can be followed, never finds anything that does not fit. However many hands are asked of `moves(() ↦ cycle(rock))`, each is a hand, so what it gives back is a `Moves`.

A type can hold itself in what a function gives back, as `Moves` does, but not in what a function takes. In `Test = test((Test) → Bool)`, whether a value is a `Test` would depend on which functions take `Tests`, so following the definition would need to know already which values are `Tests`. It would go in a circle, and a definition that goes in a circle says nothing.

Going on later added no form to the map: `Give` is an effect like any other, and `moves` and `turns` are written in the language. It added a rule about types, and widened one:

```
types    a type that holds itself: a value is of the type when following the definition,
         as far as it can be followed, never finds anything that does not fit
new
types    a type can hold itself in its contents, and in what a function gives back,
         but not in what a function takes
widened
```

18. Input and output

A request that no handler in the program answers goes outside it. Showing something on a screen, or reading what someone types, is a request answered by whatever runs the program:

```
effect Console = show : (Hand) → Unit | read : () → Hand

game = () ↦ then(show(rock), beats(rock, read()))
```

`game` shows `rock`, reads the other player's hand, and says whether rock won. Carried out, it stops at `show(rock)` and waits; whatever runs it shows rock on the screen and answers `unit`. At `read()` it waits again, for a hand. Each time the outside answers, its answer goes where the request was, and carrying out goes on: the outside works like one more handler, around the whole program, that always goes on once. Inside the language, a handler can stand in for the outside, which is how such a program is tested:

```
handle game() with
      (outcome) ↦ outcome
    | (show(a), k) ↦ k(unit)
    | (read(), k) ↦ k(scissors)

    comes to   true
```

Requests of different effects can be made together. A task that shows rock while the score is kept:

```
keep(() ↦ then(show(rock), both(round(paper, rock), round(rock, paper))), zero)
```

`keep`'s handler has no case for `show`, so `show(rock)` passes it by and goes outside: whatever runs the program shows rock and answers `unit`. Then the task goes on as in section 15, and the whole comes to `pair(false, succ(zero))`.

A player can be a person, whose rule reads the next hand from the keyboard: `player((a) ↦ read())`. That is data, but not a `Player` as section 10 defined it, since its rule asks `Console` along the way. A function type can also say what its function may ask:

```
(Hand) → Hand ! {Console}
```

This reads "from `Hand` to `Hand`, and may ask `Console`". A value is of this type when, applied to a hand, it never gets stuck, leaves no request unanswered but those of `Console`, and, with each of those answered as `Console` declares, ends, if it ends, in a hand. A rule that asks nothing, like `answer`, is one of these too: "may ask" includes not asking. So this `Player` replaces the one of section 10, and holds a person as well as a rule:

```
Player = player((Hand) → Hand ! {Console})

next(player((a) ↦ read()), rock)
```

comes to `read()`, and waits there for the person to choose.

On the map, using `E` for the effects a function may ask:

```
A ::= (A, …) → B ! E     a function type, and what it may ask; ! E left out when it lists
none     widened
E ::= {F(A, …), …}       the effects it may ask, filled; F alone if it takes none
new

types    a value is of type (A, …) → B ! E when, applied to values of types A, …, it never
gets
         stuck, leaves no request unanswered but those of the effects in E, with inputs of
the
         types they take, and, with each of those answered by a value of the type it is
answered
         with, ends, if it ends, in a value of type B
widened
names    a type or an effect is filled with as many types as it takes
widened
ends     a request that no handler answers goes outside; when the outside answers, its answer
         goes where the request was, and carrying out goes on
```

19. The whole language

These letters describe the language: `e` an expression, `v` a value, `x` a placeholder, `c` a constructor, `n` a defined name, `p` a pattern, `d` a constructor as declared, `T` a type's name, `F` an effect's name, `r` a request, `h` a handler's case, `A` and `B` types, `X` a placeholder for a type, `E` the effects a function may ask. `e, …` means any number of expressions, even none, and likewise for the other letters. `| …` after a case means more cases like it, as many as needed.

What we write: declarations and definitions, then one expression to carry out.

```
T(X, …) = d | …                          a type, with its constructors; T alone if it takes no
types
d ::= c                                  a constructor holding nothing
      c(A, …)                            a constructor holding contents of types A, …
effect F(X, …) = r : (A, …) → B | …      an effect, with its requests; F alone if it takes no
types
n = (p, …) ↦ e | …                       a definition: a function, with its name

parentheses only group: (e) is e, kept in one piece
a body reaches as far as it can
```

Names

```
each type and each effect is declared once; each constructor belongs to one type, and each
  request to one effect; each defined name is defined once
a declaration or a definition can use any of them, its own included, even one declared or
defined
  after it
in a declaration, every name in a type is a declared type, or one of the declaration's own
placeholders
the placeholders of one declaration are all different, and none has the name of a declared
type
a type or an effect is filled with as many types as it takes
in a pattern, a name is a constructor if it was declared as one, and otherwise a placeholder
the placeholders of one case are all different, and none has the name of a defined name or a
request
a placeholder means something in the body of its own case, functions inside it included
in an expression, a name means the nearest placeholder of that name, the one of the innermost
case
  around it that has one; with none, it is a constructor, a request or a defined name, and
must be one
the cases of one function all have the same number of patterns
a name written right before inputs is given as many as it takes: a defined name, as many as
its
  function takes; a constructor, as many contents as it holds, in an expression or a pattern,
and
  never none if it holds some; a request, as many as it is declared with, in an expression or
in
  a handler's case
a constructor that holds nothing is written without parentheses
```

Expressions

```
e ::= x                          a placeholder
      c                          a constructor holding nothing
      c(e, …)                    data: a constructor with its contents
      n                          a defined name
      (p, …) ↦ e | …             a function written out: its cases, tried in order
      e(e, …)                    an application
      r(e, …)                    a request
      handle e with h | …        a handler

p ::= x                          matches anything
      c                          matches only itself
      c(p, …)                    matches data built by c, with contents matching p, …

h ::= (p) ↦ e                    for the value
      (r(p, …), x) ↦ e           for a request, and the rest
```

Values

```
v ::= c                          a constructor holding nothing
      c(v, …)                    data: c holding values
      n                          a defined name: the function it is defined as
      (p, …) ↦ e | …             a function written out
```

Types

```
A ::= T(A, …)                    a type, filled; T alone if it takes none
      X                          a placeholder for a type
      (A, …) → B ! E             a function type, and what it may ask; ! E left out when it
lists none
E ::= {F(A, …), …}               effects, filled; F alone if it takes none
```

What a type means: a type is a set of values.

```
a value is of type T(A, …)           when it is built by one of T's constructors, holding
values of the
                                     types declared for its contents, with A, … in place of
T's placeholders
a value is of type (A, …) → B ! E    when, applied to values of types A, …, it never gets
stuck, leaves no
                                     request unanswered but those of the effects in E, with
inputs of the
                                     types they take, and, with each of those answered by a
value of the
                                     type it is answered with,
                                     ends, if it ends, in a value of type B

a type that holds itself: a value is of the type when following the definition, as far as
  it can be followed, never finds anything that does not fit
a type can hold itself in its contents, and in what a function gives back, but not in what
  a function takes, or following its definition would go in a circle
```

Types are definitions, and nothing in carrying out uses them. That something is of a type is a claim, which can be true or false. Claims sit on top of the language.

Steps

```
((p, …) ↦ e | …)(v, …)     becomes   the body of the first case whose patterns v, … match,
                                      with what each placeholder matched in its place
n(v, …)                    becomes   the same, with the cases of n's definition
handle v with h | …        becomes   the body of the first case for values that v matches,
                                      with what each placeholder matched in its place
handle e with h | …        becomes   the body of the first case for r whose patterns v, …
match,
                                      with what they matched in place, and the rest in place
of x,
                                      when the next thing to work out in e is a request r(v,
…),
                                      and no handler in e around that request has a case for r

the rest is (y) ↦ handle e' with h | …, where e' is e with y where the request was,
  and y is a name used nowhere in e or in the handler's cases
putting a value in place of a placeholder puts it wherever the name means that placeholder
```

Since no placeholder shares a name with a constructor, a request or a defined name, the names inside a value keep their meaning wherever it is put.

Where steps happen

```
in an application, first in what is applied, until it is a value; then in its inputs, from
left to right
in data, in its contents, from left to right
in a request, in its inputs, from left to right
in a handler, in the expression inside it
nothing happens inside a function until it is applied
```

How carrying out ends

```
in a value
waiting: the next thing to work out is a request that no handler around it has a case for;
only the
  outside can answer it, and then its answer goes where the request was, and carrying out
goes on
stuck: something is left to work out, no step applies, and it is not waiting
never: carrying out goes on forever
```

The shape: every value is data or a function, every effect is requests and a handler, and every type says which values are of a kind. Data and functions, requests and handlers are each made one way and used one way:

	made by	**used by**
data	a constructor, holding values	a pattern, which takes it apart
function	cases, each with patterns and a body	applying it to inputs
effect	a request	a handler, which answers it

Reading another language through the map. Many languages can be read as this one, with some of its forms chosen, some hidden, and their own way of writing them. Python, for example:

Python	on the map
an `Enum` class	a type, listing its constructors
`True`, `False`, an `Enum`'s members	constructors
`def`	a definition: a function with a name
`lambda`, a `def` inside a function	a function written out, using the names around it
`if`, `match`	cases
tuples, `dataclass`	constructors holding contents
`list[Hand]`	a memory holding a `List(Hand)`, at a place of its own
`Callable[[Hand], Hand]`	`(Hand) → Hand ! E`, with `E` unsaid: a Python function may ask anything
`for`, `while`	a definition that uses itself, with a memory
a variable	a memory at a place of its own
`x = x + 1`	`set(x, succ(look(x)))`, its handler hidden
`b = a`, for a list `a`	two names for one list
`class`, an object	data holding places, with functions that use them
`raise`, `try`	a request whose handler never goes on
`yield`	a request whose handler goes on later
`async`, `await`	requests whose handler holds many rests, and goes on with whichever is ready
`print`, `input`	requests the outside answers
`TypeError`	getting stuck, turned into a request that a handler can answer
a type hint on a variable or a function	a claim that it is of a type, checked by separate tools: on top of the language

Six forms of expression for saying, two more for doing, three forms of type for saying what things are, and a few rules for each. Everything else is a combination of them.